Christopher Shoo

179 Followers About Follow

Deploying django application to a production server part I

Christopher Shoo May 24, 2018 · 6 min read

setting up django with gunicorn, nginx and mysql on ubuntu server.

If you have been developing your web application with django on a development server and wondered, how i'm i going to put this into production on a real server? well you and i are going to do just that today so congratulations you are going to become a pro.

what they are.

some outline of what we are going to do.

- Installing necessary packages.
- Initial server setups.
- Preparing our django app for production.
- Setting up gunicorn.
- Setting up nginx.

Before we get started my terminal is working under /home/chris directory so change yours to that to follow-up with me. Ofcourse change chris to your username

Okay, i think we've had enough of the boring 🥱 intro, lets do some real work.

Installing necessary packages.

To follow along you should have a fresh instance of your ubuntu server with a non root user created, if you don't have access to a real server somewhere don't worry you can install a copy of ubuntu 17.10 on your local machine or virtual machine and follow along, i'm using one myself, it will do just fine.

to get started we need to install some packages, run the following commands and answer the questions to install them

```
$ sudo apt-get update
$ sudo apt-get install nginx mysql-server python3-pip python3-dev
libmysqlclient-dev ufw virtualenv
```

i am using python3 on this tutorial but if you like python2 don't worry you can always run the following commands, with just the 3 removed

```
libmysqlclient-dev ufw virtualenv
```

Wait! what did we just install?

nginx

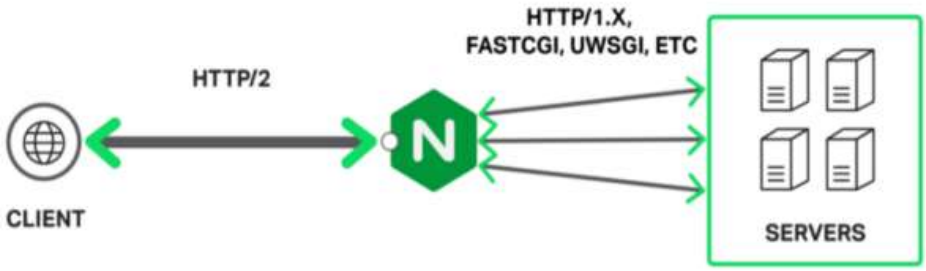

image downloaded from google

This is not a nginx tutorial but basically nginx is the web server that handles the requests from the browser and serves web pages in response to browser request. There are many other softwares like this such as apache and lighttpd, if you get familiar with things i suggest you go over and do a research on these because you might have a change of mind depending on your needs.

We've also installed mysql-server and libmysqlclient-dev, these packages will install mysql on your ubuntu and some mysql development files that are required to make things work. Again you could go with some other options such as postgresql, they all work fine, it all depends on your app requirements but avoid sqlite on a production server if you know your site is going to get a heavy traffic.

We've also installed python development files and python package manager (pip) which you must be familiar with as you are developing with django. If you haven't used it, it's used to install python packages. We're going to use virtual environment so we installed virtualenv. Virtual environments is a way we use to isolate one project packages from the other in order to avoid conflicts, that's because each app will be running on it's own environment. So you can use the old version of this package on one app and new version of the same package on onether within the same server without conflicts. Cool isn't it?

a few seconds we are going to do that.

Initial server setup.

first we are going to setup the firewall, for now we are going to block all ports and allow only port 8800 and 3306 for testing, this will later be changed when everything is setup ready. On your terminal type the following commands

```
$ sudo ufw default deny
$ sudo ufw allow 8800
$ sudo ufw allow 22
$ sudo ufw enable
```

Note that i also allowed port 22, that's in case you are connecting to the remote server with ssh while going along with the article. To see the status of the ufw type `sudo ufw status`. Now that the firewall is up and running let's setup mysql.

run the following command to setup mysql on our server

```
$ mysql_secure_installation
```

follow the steps and make sure you disallow remote login unless you have plans for it, setup new root user password. After setting up mysql we need to create the database that our django app is going to use, i'm going to call my app awesome so the database name is going to be awesome, to do that run the following commands.

```
$ mysql -u root -p
mysql> CREATE DATABASE awesome CHARACTER SET 'utf8';
mysql> CREATE USER chris;
mysql> GRANT ALL ON awesome.* TO 'chris'@'localhost' IDENTIFIED BY
'secret';
mysql> quit
```

so now we have our firewall setup, mysql is set and we already have one database on it, let's make our awesome django app and configure it to use the database we have just created. We said we are going to use virtual environment on our app so how do we do that? this way,

```
$ virtualenv venv
$ source venv/bin/activate
(env) $ pip install mysqlclient django==1.11
```

I called my virtual environment venv, you can name it anything and it will still work, to know that things went fine you will see the name of your virtual environment in brackets before the money sign on your terminal. note that i installed django too, you should also install it too but replace that 1.11 with the django version your app is running on. I asume you have your django app already so i'm not going to focus only on neccessary parts. For good practice put your django app under the same directory as your virtual environment folder, if you are following along with me put the folder containing your virtual environment under home in the <username> folder. mine is under /home/chris and it's called webapp. My awesome app has the following structure (the default one).

```
/home/chris/webapp/
            |- venv/
            |- AWESOME_PROJECT/
                 |- awesome/
                 |- AWESOME/
                    |- wsgi.py
                    |- settings.py
```

AWESOME_PROJECT is the project name and awesome is the app name. ofcourse it has all those models, url, forms files i just didn't put them as they are not neccessary here.

on your settings file change the following settings.

```
DATABASES = {
     'default': {
         'ENGINE': 'django.db.backends.mysql',
         'OPTIONS': {
             'sql_mode': 'traditional',
         },
         'NAME': 'awesome',
         'USER': 'chris',
         'PASSWORD': 'secret',
         'HOST': 'localhost',
         'PORT': '3306',
     }
 }

# direcotries for static files

STATIC_URL='/static/'
STATIC_ROOT=os.path.join(BASE_DIR, 'static/')

MEDIA_URL='/media/'
MEDIA_ROOT=os.path.join(BASE_DIR, 'media/')
```

some settings are new and some are just adjusted, for example `DEBUG=False` in order to prevent django to expose errors to our users. We also added allowed hosts `ALLOWED_HOST=['example.com',]` you need to put your domain name there if you have purchased one or your server ip address. We also changed the database dictionary field and added our own, this will shift our project from using sqlite to mysql database, if you observe you will see we have put our database info. Lastly we added our static files url where our css will live and media files url where the uploaded images and other media files will live. That's all you need to setup on your django app, after that run the migration commands.

```
(env) $ python manage.py makemigrations
(env) $ python manage.py migrate
(env) $ python manage.py collectstatic
```

you can run the app on the development server to test if things are still working fine.

okay enough of the first part. The next last part we are going to setup nginx and gunicorn and that will be all. I'm going to the last cool part, things get soo much cooler there, if yo want to come with me click the below link, and feel free to respond or add few claps, that will motivate me to write more.

https://medium.com/@_christopher/deploying-my-django-app-to-a-real-server-part-ii-f0c277c338f4

Python Nginx MySQL Servers Django